

AnaTele Illustrated Guide

Learn drone flight-log investigation from scratch

Avinash Chandra Gupta

`https://avinashistoocool.github.io/AnaTele/learn/`

Every chart is drawn from real simulator flights analysed by AnaTele itself.
Generated from the web guide by `docs/figures/build_guide_tex.py`.

Contents

1 Introduction **4**

1.1 How to use this course 4

2 Drones 101 **5**

2.1 The parts 5

2.2 Three angles: roll, pitch, yaw 6

2.3 Thrust, weight and hover 6

3 Logs and MAVLink **8**

3.1 What is a flight log? 8

3.2 A log is a stream of typed messages 9

3.3 Different messages, different rates 10

3.4 What AnaTele keeps 10

4 Time and sources **11**

4.1 Two clocks 11

4.2 Epochs 11

4.3 Sources 12

4.4 Pairing samples 12

5 Flight phases **13**

5.1 Step 1: the armed baseline 13

5.2 Step 2: confirm, then backtrack 14

5.3 Step 3: the pre-event window 14

6 Attitude and tilt error **16**

6.1 Desired versus actual 16

6.2 One number: the tilt error 16

6.3 What it looks like in a failure 18

7 Motors and mixing **19**

7.1 The mixer 19

7.2 Healthy motors . . . 20
7.3 A failing motor . . . 20
7.4 Saturation . . . 21
7.5 Imbalance . . . 22

8 Vibration, EKF and compass 23
8.1 Vibration . . . 23
8.2 The EKF: the autopilot's best guess . . . 24
8.3 Compass interference . . . 24

9 Rules and their four states 26
9.1 What a rule is . . . 26
9.2 Persistence: ignore blips, catch trends . . . 26
9.3 The four states . . . 27
9.4 Counting the five simulated flights . . . 28

10 Hypotheses 29
10.1 The four built-in hypotheses . . . 29
10.2 The score . . . 29
10.3 Worked example: the motor-failure flight . . . 30

11 Missions and geometry 31
11.1 Where did it go? . . . 31
11.2 Distance on a sphere . . . 32
11.3 When does a waypoint count as reached? . . . 32
11.4 Drawing the path . . . 33

12 The simulator 34
12.1 How it works . . . 34
12.2 The five scenarios . . . 35

13 Sharing with .ant files 36
13.1 What is inside . . . 36
13.2 Saving and opening . . . 37
13.3 The report inside . . . 37

A Glossary 38

Chapter 1

Introduction

This is an illustrated course. It assumes **no** background: you don't need to have flown a drone, opened a log or seen the maths before. By the end you'll be able to read an AnaTele report and explain every number in it.

Every chart is drawn from **real data**: flights from AnaTele's built-in simulator, analysed by the same code you run. The script that draws them is `docs/figures/make_guide_figures.py`. Some pages also have small interactive widgets that run in your browser.

1.1 How to use this course

- **In a hurry?** Read chapters 1, 4, 5 and 9. That's enough to read a crash report.
- **Investigating a real crash?** Do chapters 4–9 next to the Diagnostics reference.
- **Writing your own rules?** Read chapters 8 and 9, then Writing a rule.

The one idea to keep in mind

AnaTele reports **observations** and **hypotheses**. It never gives a crash verdict or a probability of cause. If data is missing, the answer is *unknown*, never *fine*.

Chapter 2

Drones 101

2.1 The parts

A quadcopter is four motors with propellers on arms, a battery, and a small computer called the **flight controller**. **AnaTele works with flight controllers running** ArduPilot**, the open-source autopilot used in everything from hobby quads to survey drones.

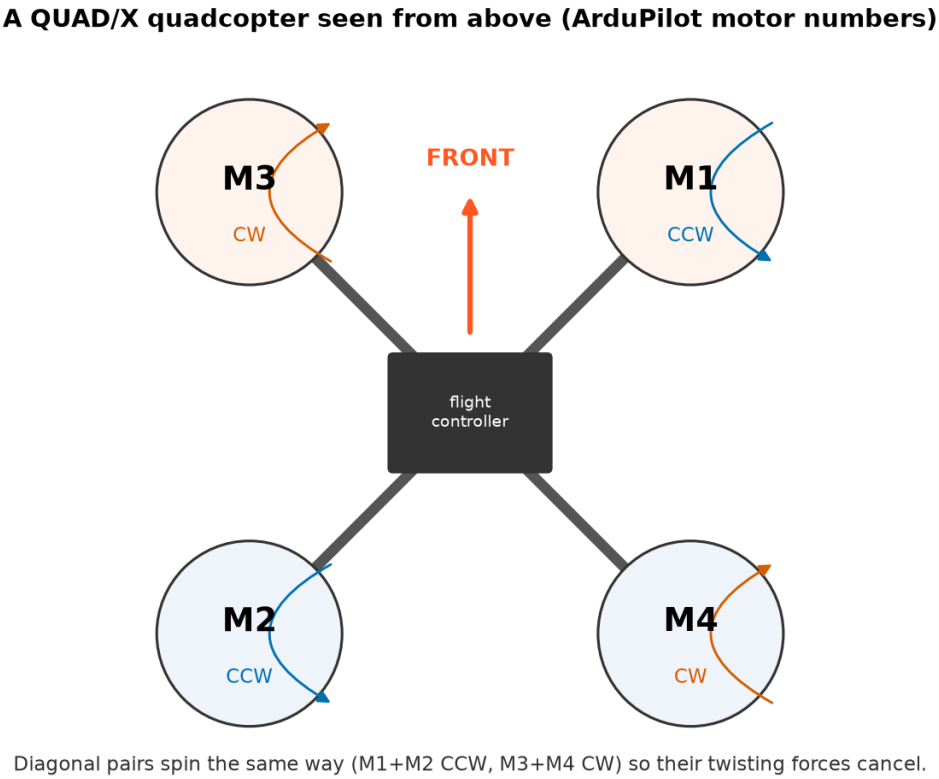


Figure 2.1: QUAD/X layout

ArduPilot numbers the motors of an "X" quad the way the figure shows. Motor 1 is front-right, not front-left, which surprises most people. Diagonal pairs spin the same way: M1 and M2 counter-clockwise (CCW), M3 and M4 clockwise (CW). Each spinning propeller twists the frame the opposite way. With two of each direction, the twists cancel and the drone doesn't spin.

Why this matters in an investigation

If a motor is wired to the wrong output, or a propeller is on the wrong motor, the drone usually flips on takeoff. AnaTele reads `SERVOn_FUNCTION`, `FRAME_CLASS` and `FRAME_TYPE` from the log so that every motor number it shows matches this picture.

2.2 Three angles: roll, pitch, yaw

The **attitude** of a drone is which way it's facing and how it's tilted. Three angles describe it:

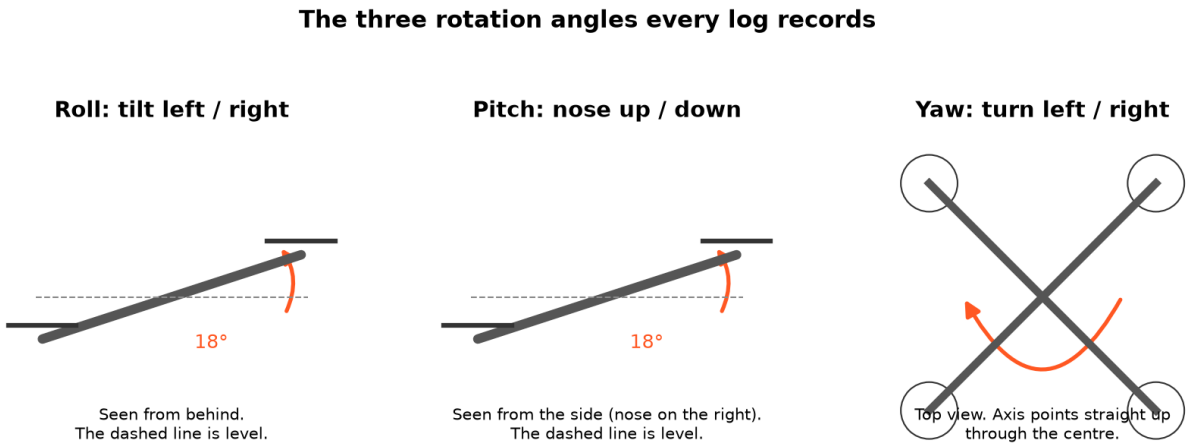


Figure 2.2: Roll, pitch and yaw

- **Roll**: tilting left or right, like a plane banking.
- **Pitch**: nose up or nose down.
- **Yaw**: turning left or right while staying level. This is the drone's *heading*.

A quadcopter has no rudder or wings, so it moves by **tilting**. To fly forward it pitches its nose down. Part of the thrust then points backwards and pushes it forward. That is why "is the tilt correct?" is the most important question in chapter 5.

2.3 Thrust, weight and hover

To hover, the motors must push up exactly as hard as gravity pulls down.

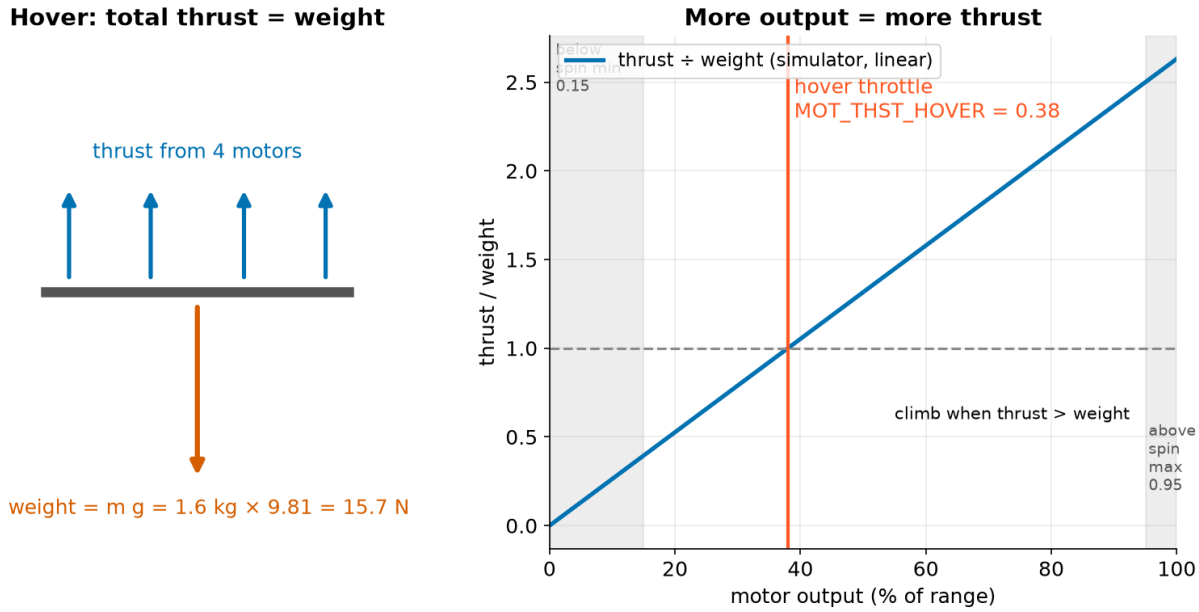


Figure 2.3: Thrust balance

The simulator's drone weighs 1.6 kg, so its weight is

$$W = mg = 1.6\,\text{kg} \times 9.81\,\text{m/s}^2 \approx 15.7\,\text{N}.$$

ArduPilot records the motor output needed to hover as `MOT_THST_HOVER`. In the simulated flights it is **0.38**: 38 % of the usable range. More output gives more thrust and the drone climbs. Less output and it descends.

Motors also have limits. Below `MOT_SPIN_MIN` (0.15 here) they can stall. Above `MOT_SPIN_MAX` (0.95) there's no headroom left. A motor that sits at one of these limits is **saturated**. Chapter 6 shows why that's an important clue.

Check your understanding

The drone is hovering and you want it to roll right. Which motors must push harder?

Answer

The **left** ones (M2 back-left and M3 front-left). Pushing up harder on the left tips the drone to the right.

Chapter 3

Logs and MAVLink

3.1 What is a flight log?

During a flight, the flight controller reads its sensors hundreds of times a second. It estimates where it is, decides what to do and drives the motors. It also writes a record of all of this, and that record is the **flight log**.

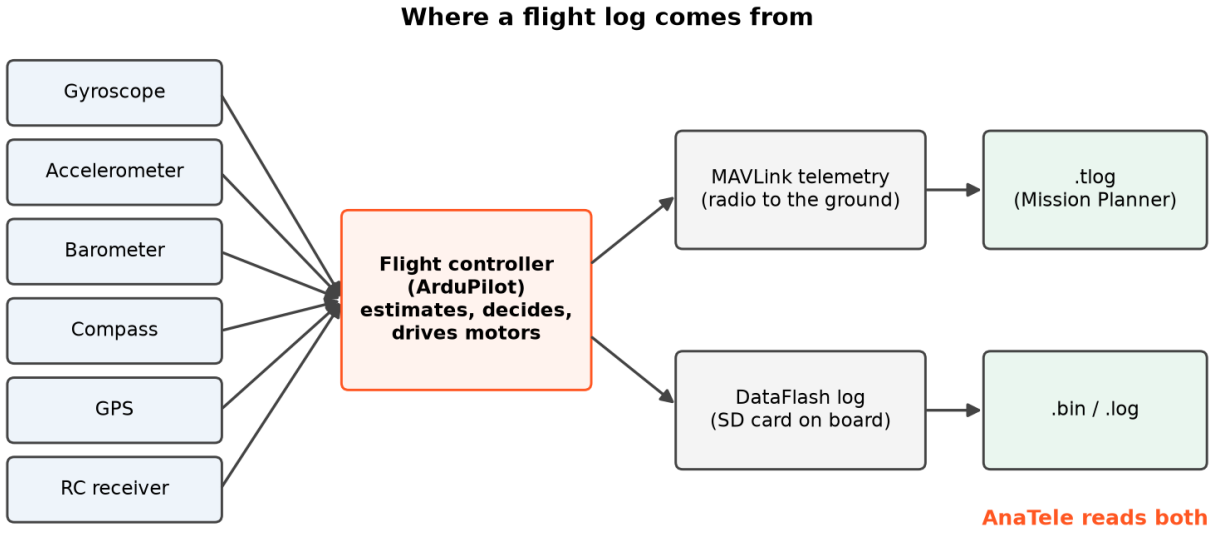


Figure 3.1: Where a log comes from

There are two kinds, and AnaTele reads both:

Log	Where it is made	What it contains
Telemetry log `.tlog`	On the ground station (e.g. Mission Planner), from the radio link	**MAVLink** messages, each stamped with the time the ground received it
DataFlash log `.bin` / `.log`	On the drone's SD card	ArduPilot's own on-board records; usually higher rate and more detail

MAVLink is the language drones and ground stations use to talk. Each MAVLink message has a type (for example `ATTITUDE` or `GPS_RAW_INT`) and named fields (`roll`, `pitch`, `lat`, `lon`...).

3.2 A log is a stream of typed messages

Here is the start-to-finish stream of a simulated flight in which one motor fails. Each tick is one message.

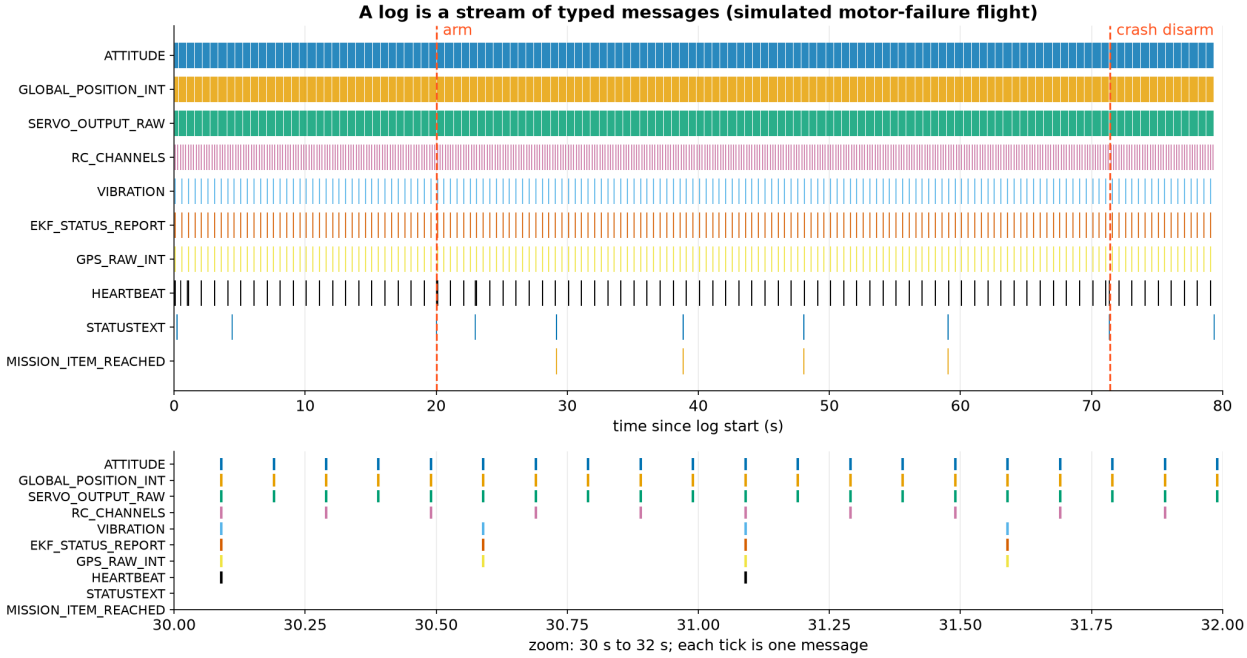


Figure 3.2: Message stream

Things to notice:

- `ATTITUDE`, `GLOBAL_POSITION_INT` and `SERVO_OUTPUT_RAW` are dense bands: they arrive many times a second.

- `STATUSTEXT` (human-readable text such as *"Crash: Disarming"*) shows up only a handful of times, but these messages are often the most important ones.

- `MISSION_ITEM_REACHED` marks each waypoint the autopilot reports reaching.

3.3 Different messages, different rates

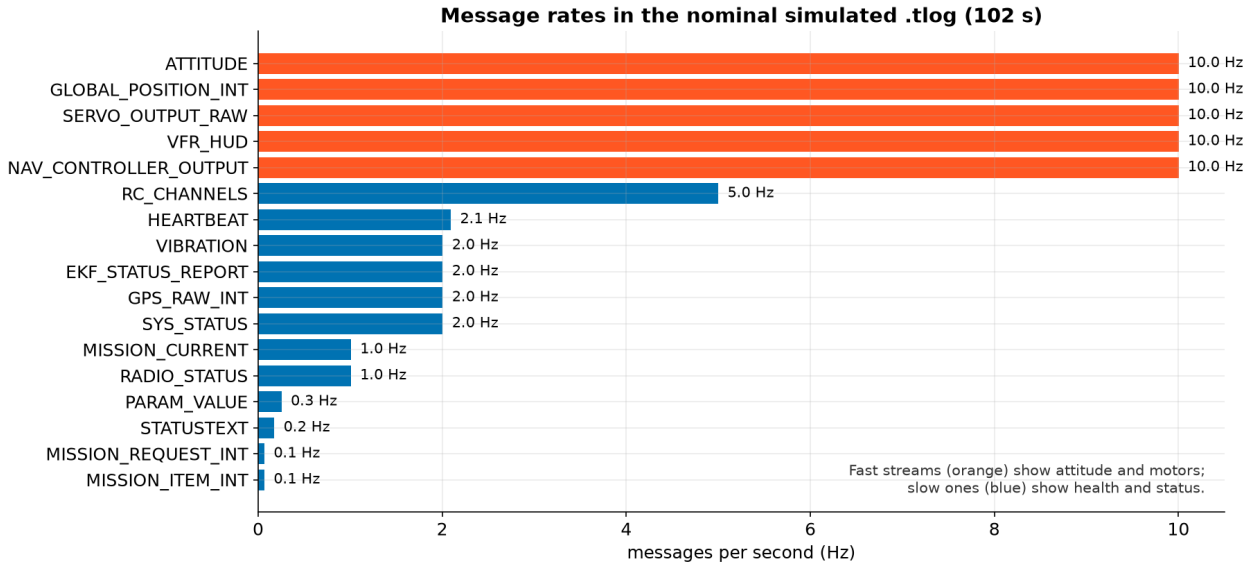


Figure 3.3: Sample rates

In this 102-second simulated `.tlog`, attitude, position, motor outputs and the controller's targets arrive at **10 Hz** (10 times a second). RC input arrives at 5 Hz, and vibration, EKF status, GPS and battery at **2 Hz**.

Rate limits what you can conclude

A rule that needs "above 30 for at least 0.5 s" can't be checked properly with data every 0.5 s. AnaTele knows this. When the data is too sparse, it marks the rule **unassessable** instead of pretending it's healthy (chapter 8).

3.4 What AnaTele keeps

When AnaTele opens a log it keeps **every decoded record**, with its byte offset in the file. The simulated motor-failure log has 5,387 records. Every chart, threshold and hypothesis in the report can therefore be traced back to the exact bytes it came from. This is the *evidence viewer* in the desktop app.

Chapter 4

Time and sources

4.1 Two clocks

Every record in a log has a time, but not always from the same clock.

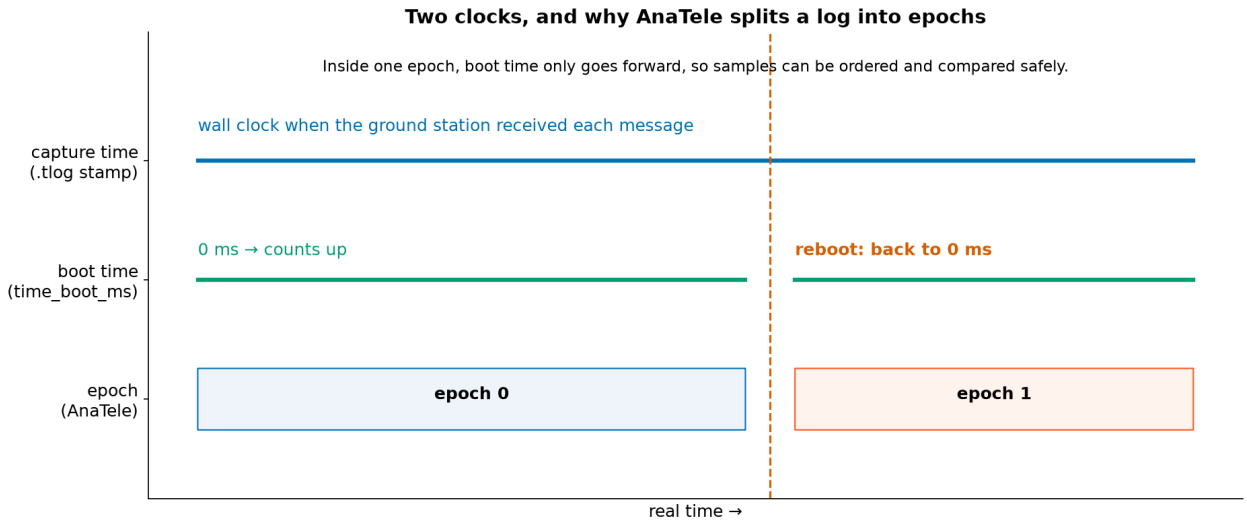


Figure 4.1: Time bases

- **Capture time**: when the ground station received the message (the 8-byte stamp in front of each `.tlog` record). It's wall-clock time, so it can jump if the PC's clock is adjusted, and it includes radio delay.
- **Boot time** (`time_boot_ms`, `time_usec`): how long the flight controller has been powered on. It's precise, but it **restarts from zero** when the board reboots.

4.2 Epochs

If the board reboots mid-log, "10 s after boot" happens twice. Put both runs on one timeline and you'd mix two different flights. AnaTele avoids this by splitting the log into **epochs**. A new epoch starts whenever a clock clearly goes backwards. Inside one epoch, time only moves forward, so samples can be ordered, paired and compared safely.

The exact tests (per source) are:

Clock	New epoch when…	Why the margin
`.tlog` boot time `time_boot_ms`	it drops by more than **10 s**	Small regressions are just reordered / delayed packets
DataFlash boot time	it drops by more than **1 s**	On-board records are written in order
`.tlog` capture time	it drops by more than **2 s**	PC clock adjustments separate the whole capture

A boot-time drop from near 2^{32} ms back to near zero is recognised as a counter **wrap** (after ~49.7 days), not a reboot. Every epoch split is listed as a diagnostic, so you can see why it happened.

4.3 Sources

A capture can hold messages from more than one system: the drone, a gimbal, the ground station, a companion computer. MAVLink identifies each sender by a *system id* and *component id*. AnaTele calls each pair a **source** (for example `1:1` is the autopilot) and never mixes signals from different sources.

4.4 Pairing samples

Many checks compare two signals, for example desired and actual roll. They don't arrive at exactly the same instant, so AnaTele pairs each sample with the **nearest** sample of the other signal, within a tolerance. For attitude that tolerance is **0.25 s**. If no partner is close enough, the pair is skipped rather than guessed.

Design principle

Nothing is interpolated or invented. A derived value always comes from real samples, close enough in time to be compared, and it keeps a link to the record it came from.

Chapter 5

Flight phases

A crash investigation is about **timing**: what was normal, when did it stop being normal, what came first. So the first thing AnaTele does with each armed period (an *episode*) is split it into phases.

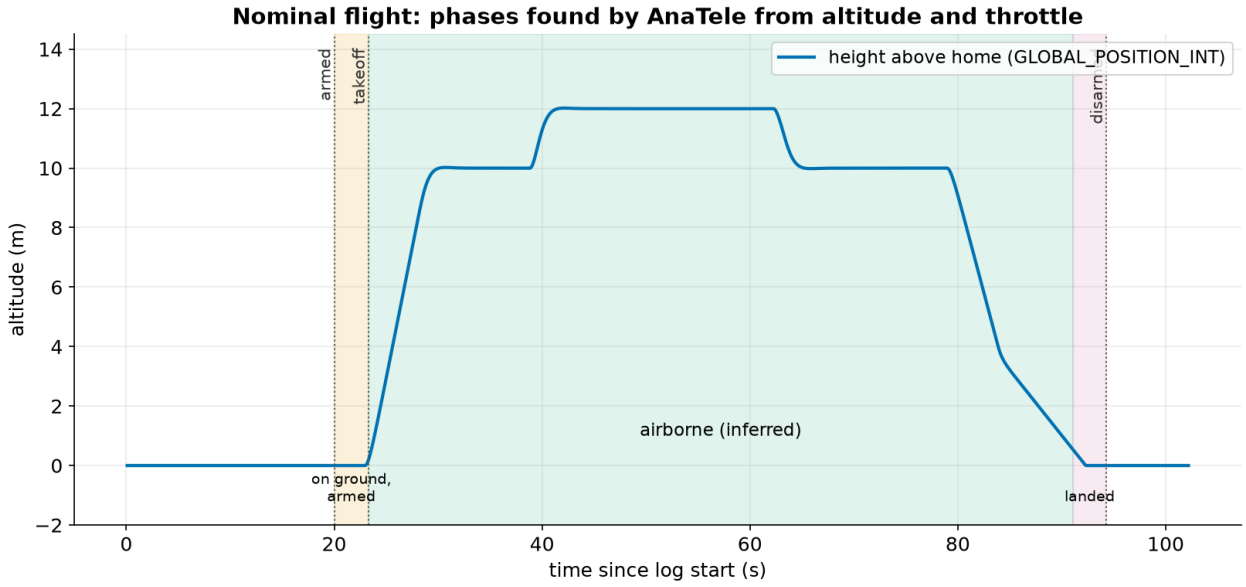


Figure 5.1: Altitude and phases

In the nominal simulated flight, the drone arms at 20.0 s and AnaTele finds takeoff at **23.29 s**. It's airborne until 91.09 s and disarms at 94.27 s.

5.1 Step 1: the armed baseline

Altitude estimates drift, so "altitude = 0" doesn't reliably mean "on the ground". AnaTele takes the **median** home-relative altitude during the first **2 s** after arming as the ground level (the *baseline*). The median is used instead of the mean so a single bad sample can't move it.

5.2 Step 2: confirm, then backtrack

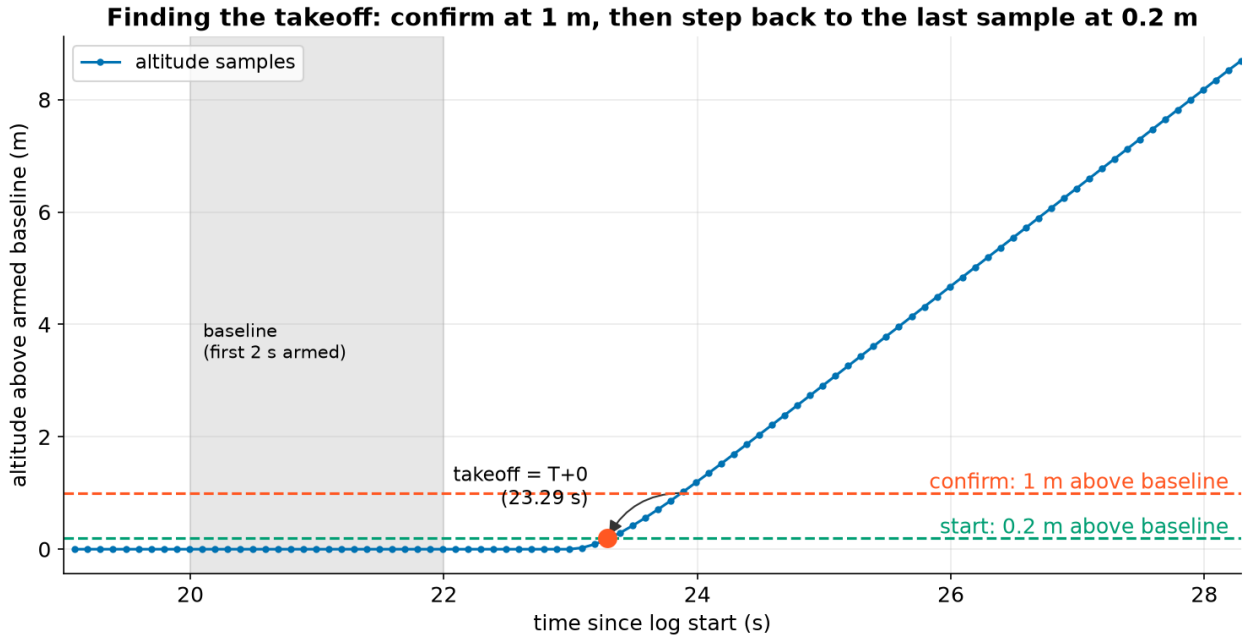


Figure 5.2: Takeoff detection

1. Find the first sample at least **1.0 m** above the baseline. That is clearly flying, not noise.

2. Check that there was real **demand** in the 5 s before it: throttle at least **10 %**, or (if throttle isn't logged) a motor output of at least 25 % of its range. If the altitude rose with no demand, it's treated as estimate drift and the drone is *not* considered airborne.

3. Walk **backwards** to the last sample still within **0.2 m** of the baseline. That's the takeoff time, **T+0**.

If the autopilot itself reports "Taking off" or "In air" (a landed-state message), AnaTele uses that instead and labels the phase *reported* rather than *inferred*.

5.3 Step 3: the pre-event window

Rules mostly look at the time **before** anything went wrong. Once the drone has hit the ground, all the sensors go wild, and that's a consequence, not a cause.

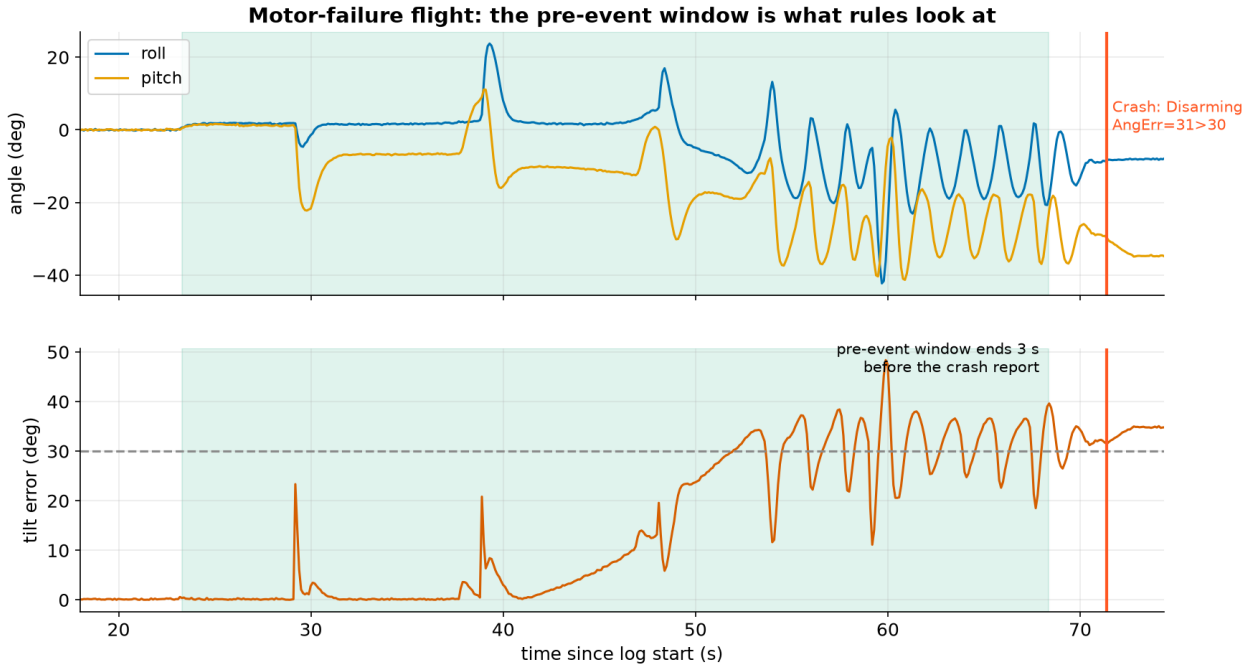


Figure 5.3: Upset detection

The window ends at the earliest of:

- the first moment roll or pitch exceeds **60°** (an attitude excursion, i.e. loss of control);
- **3 s** before an autopilot crash report (the *event guard*);
- the last sample at least 0.5 m above the baseline.

In the motor-failure flight, ArduPilot reported *"Crash: Disarming: AngErr=31>30"* at 71.37 s. The roll and pitch never passed 60°, so the pre-event window runs from takeoff (23.29 s) to **68.37 s**, exactly 3 s before the report.

Phase labels in the report

- `airborne_reported`: from the autopilot's own landed-state messages.
- `airborne_inferred`: from altitude rise plus demand, as above.
- `no_airborne_evidence`: armed, but never clearly flew.
- `unassessable`: no altitude data, so we can't tell.

Chapter 6

Attitude and tilt error

6.1 Desired versus actual

The autopilot constantly decides what attitude it **wants** (desired roll and pitch). It then tries to achieve it with the motors. A healthy drone follows closely:

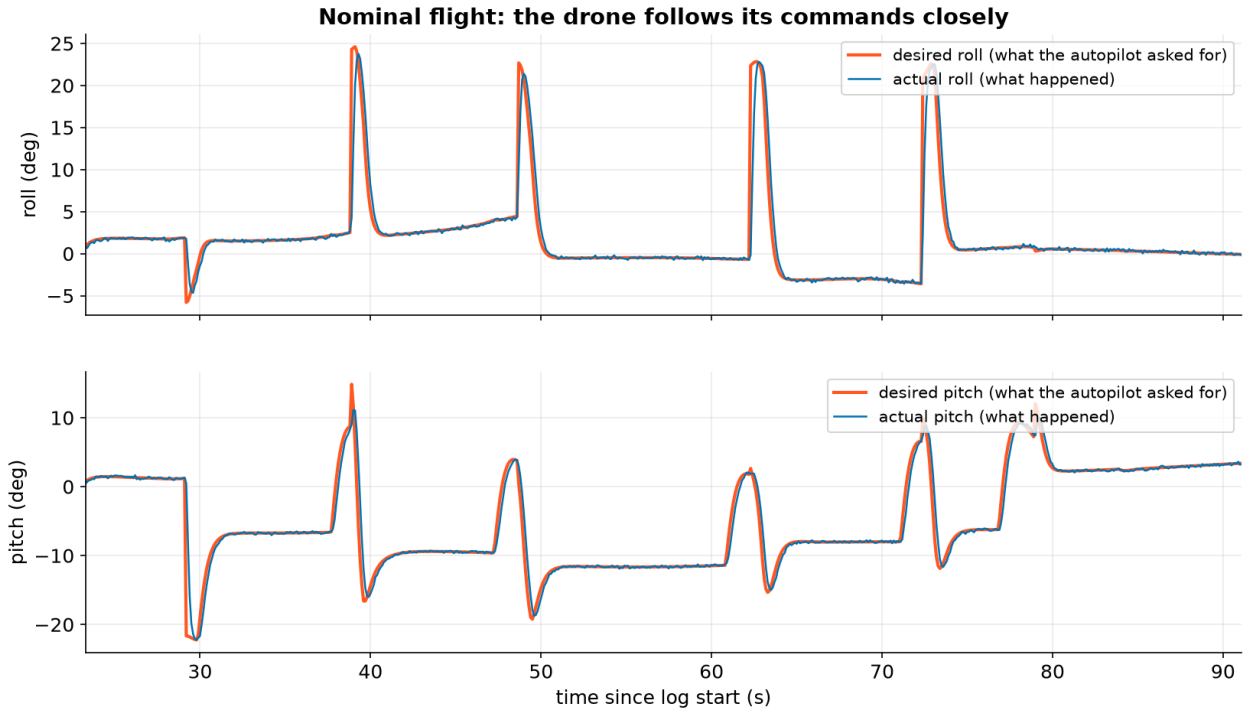


Figure 6.1: Desired vs actual

When a motor is failing, a propeller is damaged or the frame is wrongly configured, the drone can no longer follow. The gap between "wanted" and "got" grows. This is one of the strongest clues in an investigation.

6.2 One number: the tilt error

Roll and pitch errors could be checked separately, but what really matters is **which way the thrust points**. So AnaTele turns each (roll, pitch) pair into a unit vector along the drone's thrust axis:

$$\hat{t}(\phi, \theta) = (\sin\theta\cos\phi,\ -\sin\phi,\ \cos\theta\cos\phi)$$

where ϕ is roll and θ is pitch. The **tilt error** is the angle between the desired and actual vectors:

$$e = \arccos(\hat{t}_{\text{desired}} \cdot \hat{t}_{\text{actual}}).$$

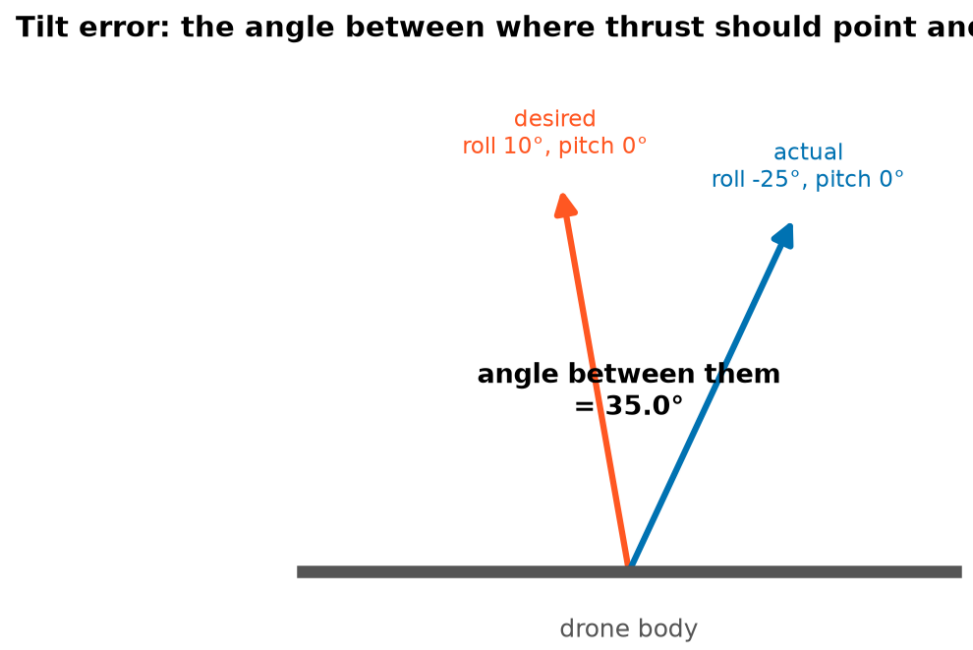


Figure 6.2: Tilt error geometry

Desired and actual samples are paired with the nearest-in-time rule from chapter 3 (within 0.25 s).

Interactive

This page has an interactive *tilt-error* widget in the web version: `https://avinashistoocool.github.io/AnaTele/learn/05-attitude-and-tilt-error/`

6.3 What it looks like in a failure

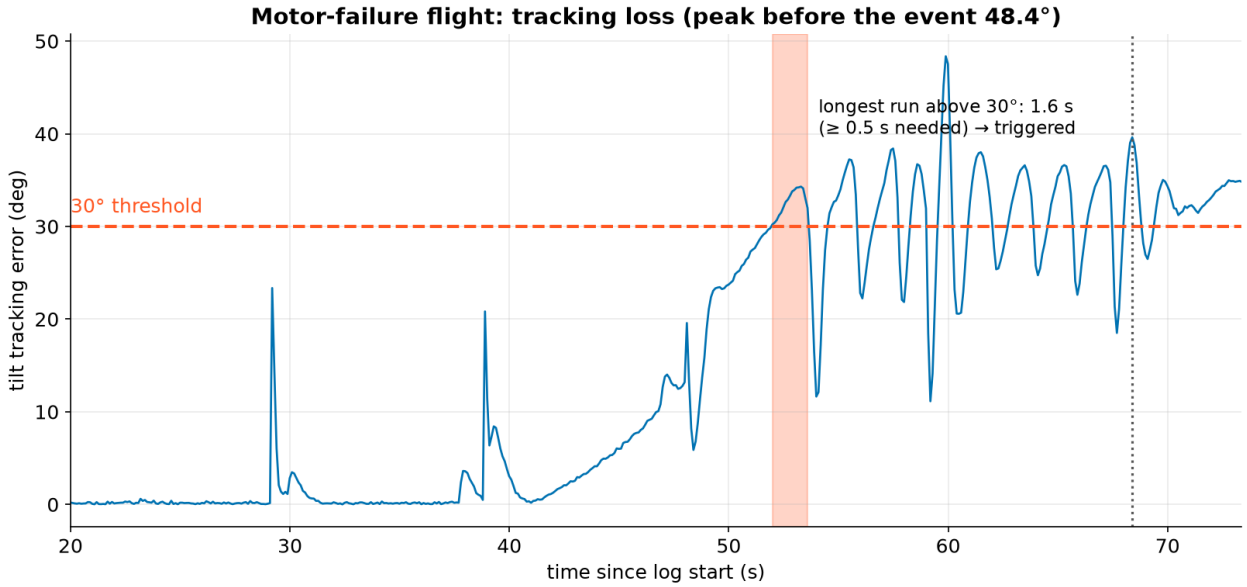


Figure 6.3: Tilt error over time

In the nominal flight the tilt error never exceeds **23.4°**, during brisk turns. In the motor-failure flight it first passes 30° at **51.99 s** and peaks at **48.4°** before the event window ends. The longest stretch above 30° lasts **1.6 s**. The `tracking-loss` rule needs **30° for at least 0.5 s**, so it is **triggered**.

Tilt error is a symptom, not a cause

A large tilt error says "the drone could not do what it was told". It doesn't say why. A weak motor, a loose prop, the wrong frame type, a very aggressive tune or a strong gust can all cause it. AnaTele uses it as evidence for the *thrust/configuration* hypothesis (chapter 9), together with the motor signals in chapter 6.

Chapter 7

Motors and mixing

7.1 The mixer

The pilot or autopilot asks for four things: **throttle** (up/down), **roll**, **pitch** and **yaw**. The **mixer** turns these into one command per motor.

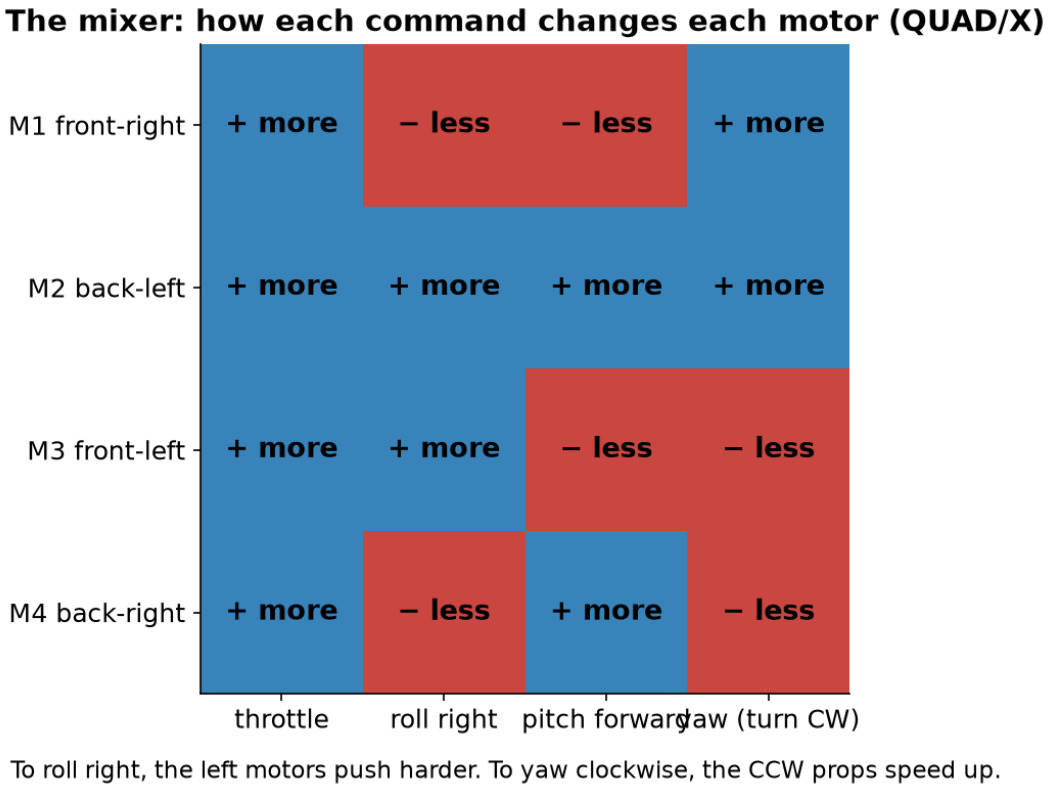


Figure 7.1: Mixer matrix

For each motor at arm position (x, y) (right, forward) with spin s (+1 for CCW, −1 for CW), the command is roughly

$$u_i = \text{throttle} \; - \; x_i \, \text{roll} \; + \; y_i \, \text{pitch} \; + \; s_i \, \text{yaw}.$$

Each command must stay between `MOT_SPIN_MIN` and `MOT_SPIN_MAX`. If the sum doesn't fit, ArduPilot (and AnaTele's simulator) keeps roll and pitch, because they keep the drone upright, and gives

up yaw first.

Try it: push the roll slider until one motor hits a limit.

Interactive

This page has an interactive *mixer* widget in the web version: `https://avinashistoocool.github.io/AnaTele/learn/06-motors-and-mixing/`

7.2 Healthy motors

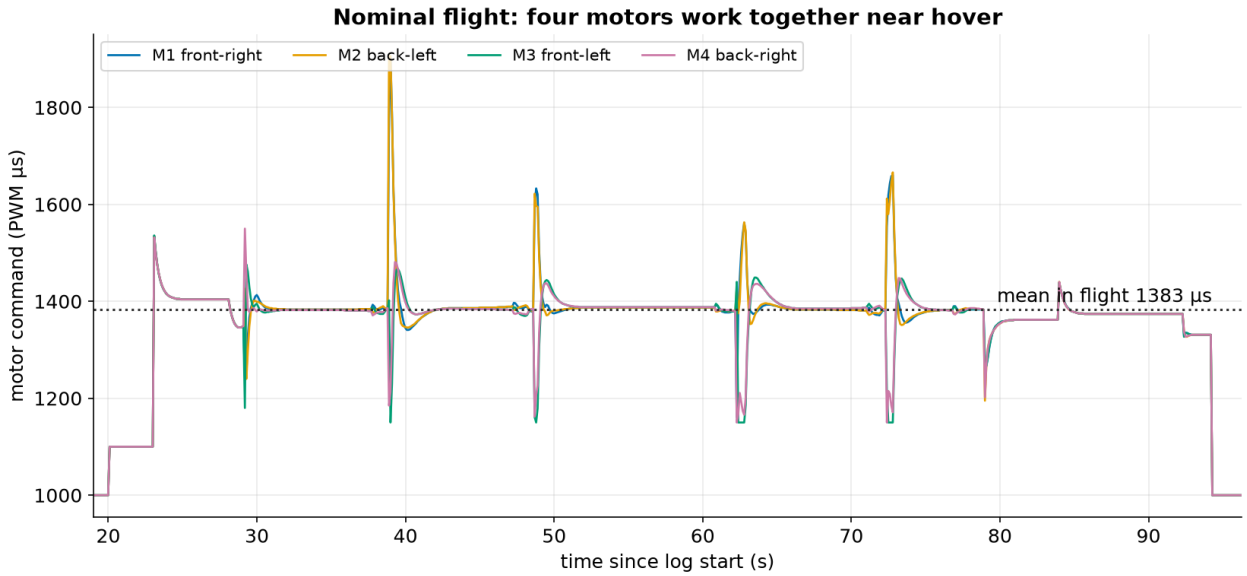


Figure 7.2: Nominal motor outputs

In the nominal flight all four commands move together around a mean of about **1383 µs** while airborne. They spread apart briefly in turns and accelerations.

7.3 A failing motor

In the motor-failure scenario, the simulator makes motor 3 lose thrust. Its health falls as $h_3 = \max(0.3,\ 1 - 0.06\,(t - t_f))$ from $t_f = 40.85$ s. The autopilot doesn't know this. It only sees the drone tilting the wrong way, so it **commands motor 3 harder** and the opposite motor (4) softer.

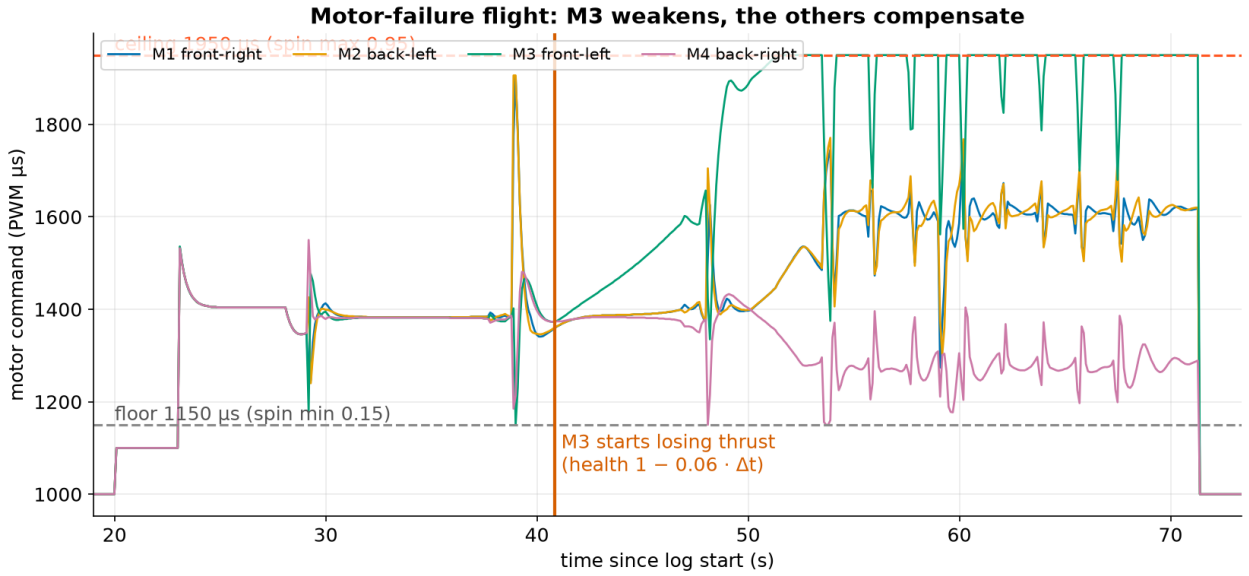


Figure 7.3: Motor outputs in failure

Read this the right way round

The motor with the **highest** command is often the **weak** one. The controller is trying to make up for missing thrust. A log records *commands*, not thrust, so AnaTele says "command anomaly", never "motor 3 failed".

7.4 Saturation

A motor is **at ceiling** if its PWM is within 2 µs of the ceiling, and **at floor** if within 2 µs of the floor. The limits come from the log's own parameters:

$$\text{floor} = \text{PWM}_{\text{min}} + \text{SPIN}_{\text{min}}\,(\text{PWM}_{\text{max}} - \text{PWM}_{\text{min}})$$

$$\text{ceiling} = \text{PWM}_{\text{min}} + \text{SPIN}_{\text{max}}\,(\text{PWM}_{\text{max}} - \text{PWM}_{\text{min}})$$

With 1000–2000 µs and spin limits 0.15/0.95, that's **1150 µs** and **1950 µs**.

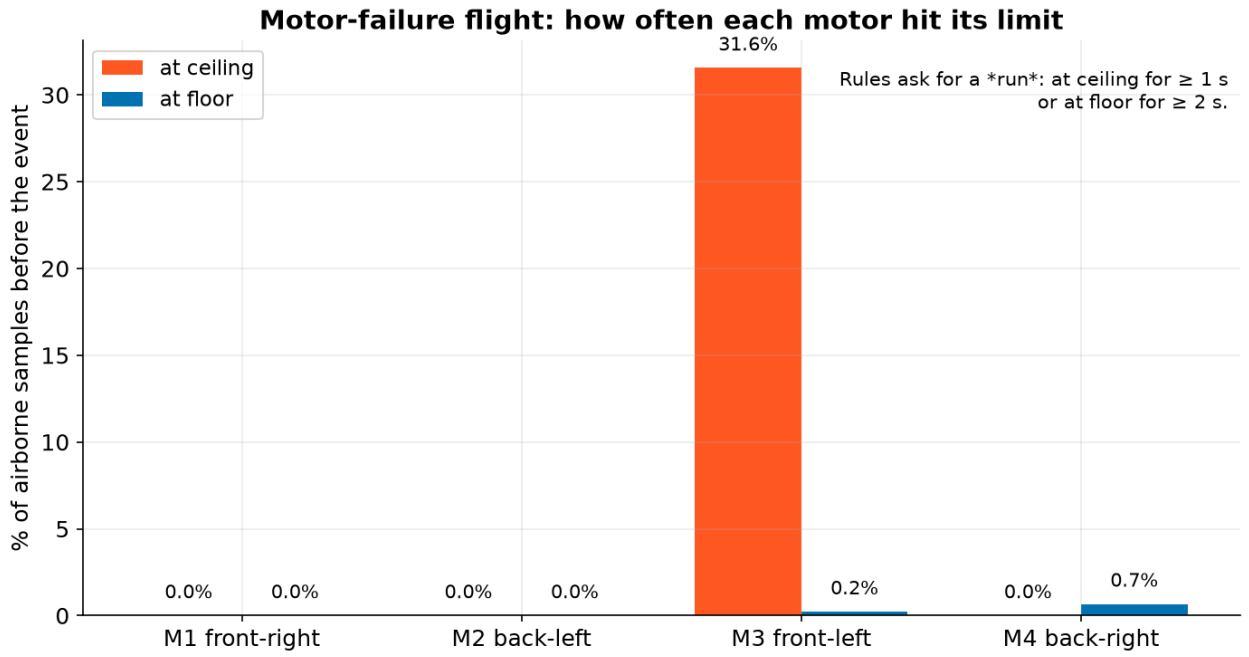


Figure 7.4: Saturation fractions

Before the event, motor 3 spends **31.6 %** of the airborne samples at its ceiling. Its longest unbroken stretch is **2.2 s**. The `motor-ceiling-saturation` rule needs at least 1 s, so it's **triggered**. No motor stays at the floor for 2 s, so `motor-floor-saturation-pre-event` is not triggered.

7.5 Imbalance

AnaTele also averages each motor's command over the pre-event window (as a fraction of the PWM range). In the failure flight the means are M1 0.47, M2 0.47, **M3 0.65**, **M4 0.34**: a spread of **0.31**, above the 0.12 screening limit. `motor-output-imbalance-pre-event` is triggered. Wind, payload offset and yaw demand can also cause imbalance, and the report says so.

Chapter 8

Vibration, EKF and compass

8.1 Vibration

The accelerometers measure acceleration, which the autopilot needs to know which way is down. A damaged propeller, loose motor or stiff mounting adds shaking that the accelerometers also pick up. Too much of it and the position and altitude estimates degrade.

ArduPilot logs vibration levels in m/s^2 for each axis. Its own guidance treats **30** as worth looking at and **60** as bad.

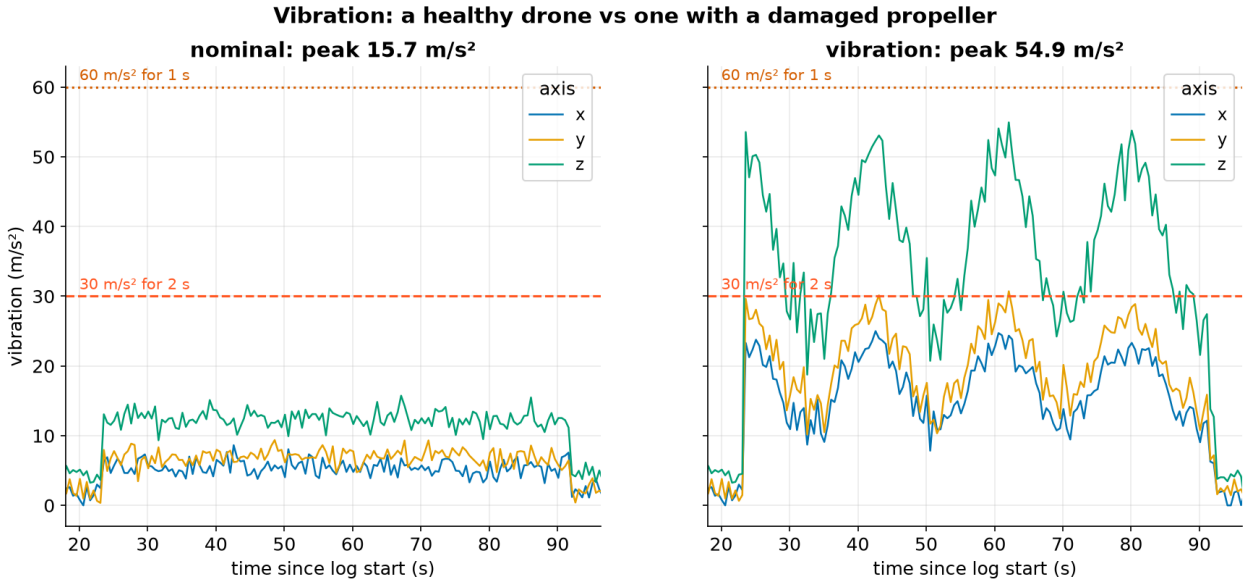


Figure 8.1: Vibration comparison

Flight	Peak vibration	Longest run above 30 m/s^2 (pre-event)	Rule
nominal	15.7 m/s^2	0 s	not triggered
vibration (damaged prop)	54.9 m/s^2	**12.5 s** on z	**triggered**

`vibration-persistent-pre-event` triggers if any axis stays above **30 for 2 s** or above **60 for 1 s**. The damaged-prop flight also logged **283 accelerometer clipping events**, where the sensor hit its maximum, so `clipping-pre-event` triggers too.

8.2 The EKF: the autopilot's best guess

ArduPilot fuses all its sensors in an **Extended Kalman Filter (EKF)**. For each sensor it predicts what the next reading should be, then compares that with the actual reading. The gap, scaled by how big a gap it expects, is the **test ratio**:

$$\text{ratio} = \frac{\text{innovation}^2}{\text{expected variance}} \quad \text{(reported so that 1.0 = at the limit).}$$

Below 1 means the sensor agrees with the filter. Above 1 means the EKF is starting to distrust it.

8.3 Compass interference

High motor currents create magnetic fields. If the compass sits too close to the power wires, its heading shifts whenever the motors work hard.

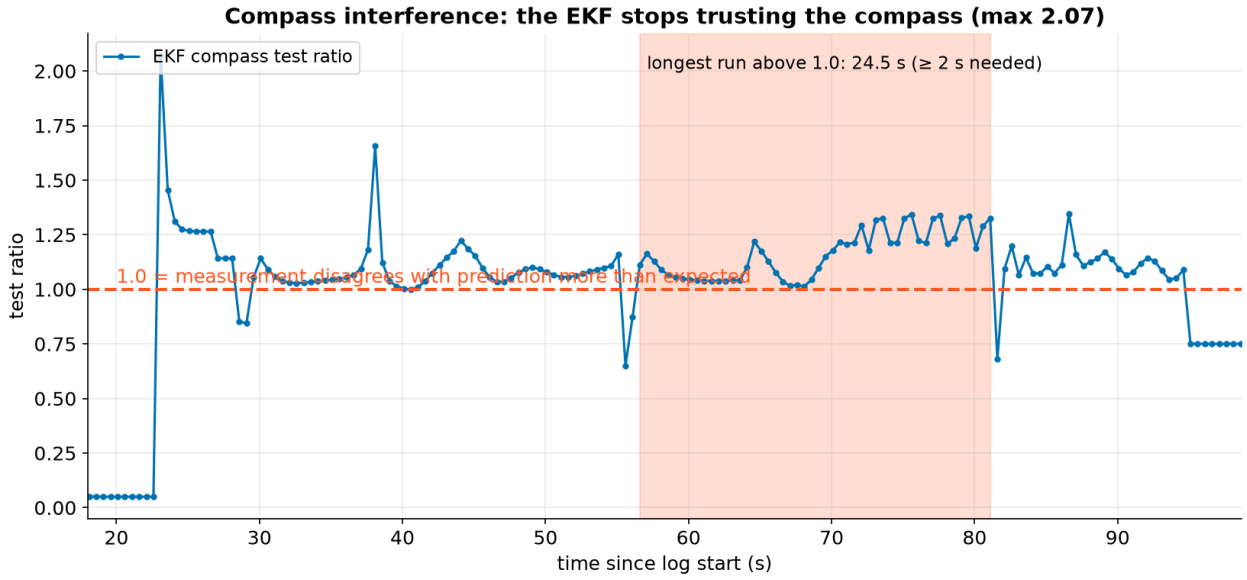


Figure 8.2: EKF compass ratio

In the compass-interference flight, the EKF compass ratio reached **2.07**. Before the event it stayed above 1.0 for **24.5 s** in one run. `ekf-compass-innovation-pre-event` needs **above 1.0 for 2 s**, so it's **triggered**.

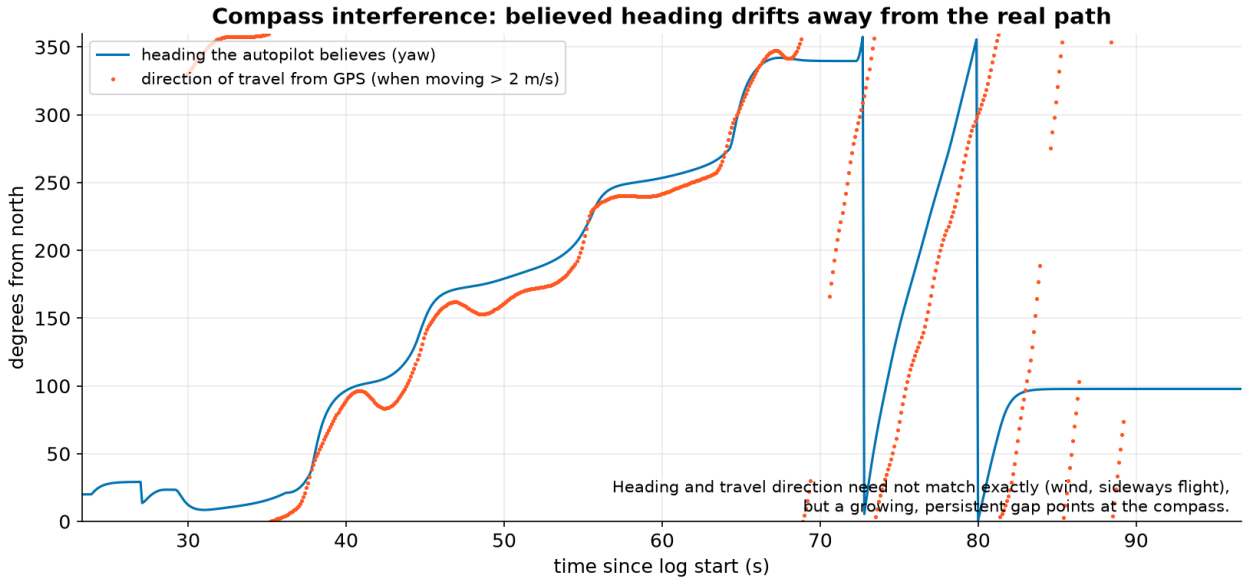


Figure 8.3: Heading disagreement

When the heading is wrong, the drone flies in curving, growing circles ("toilet-bowling"). The autopilot steers for where it *thinks* it's facing, not where it actually is.

Why a 'compasses inconsistent' message after a crash proves little

After the motor-failure crash, the simulator (like real ArduPilot) printed *"PreArm: Compasses inconsistent"*. A crash can bend a compass mount, so this message comes *after* the incident. AnaTele records it (`compasses-inconsistent-reported`) but gives it little weight.

Chapter 9

Rules and their four states

9.1 What a rule is

A rule is a small, versioned, written-down check: "if *this signal* stays above *this threshold* for *this long*, during *this phase*, report it". All rules live in one file, the **rule pack** (`telemetry_platform/rules/core.yaml`). Every report records which pack version and SHA-256 it used, so the same log always gives the same answer.

9.2 Persistence: ignore blips, catch trends

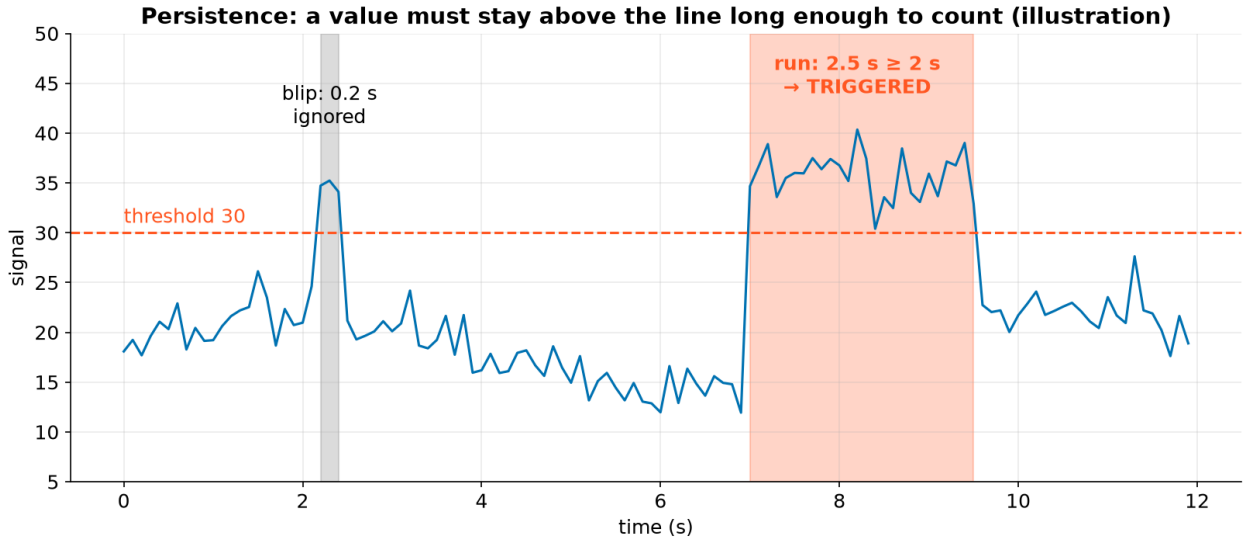


Figure 9.1: Persistence

Real signals are noisy. A single spike is usually meaningless, so most rules need a **run**: consecutive samples above the threshold, lasting at least `for_s` seconds. Two more details:

- If two samples are too far apart (more than the **maximum gap**), the run is broken. AnaTele doesn't assume what happened in between.
- A run's length is measured from its **first to its last** sample above the threshold.

Try it, including what happens when data gets sparse:

Interactive

This page has an interactive *persistence* widget in the web version: `https://avinashistoocool.github.io/AnaTele/learn/08-rules-and-states/`

9.3 The four states

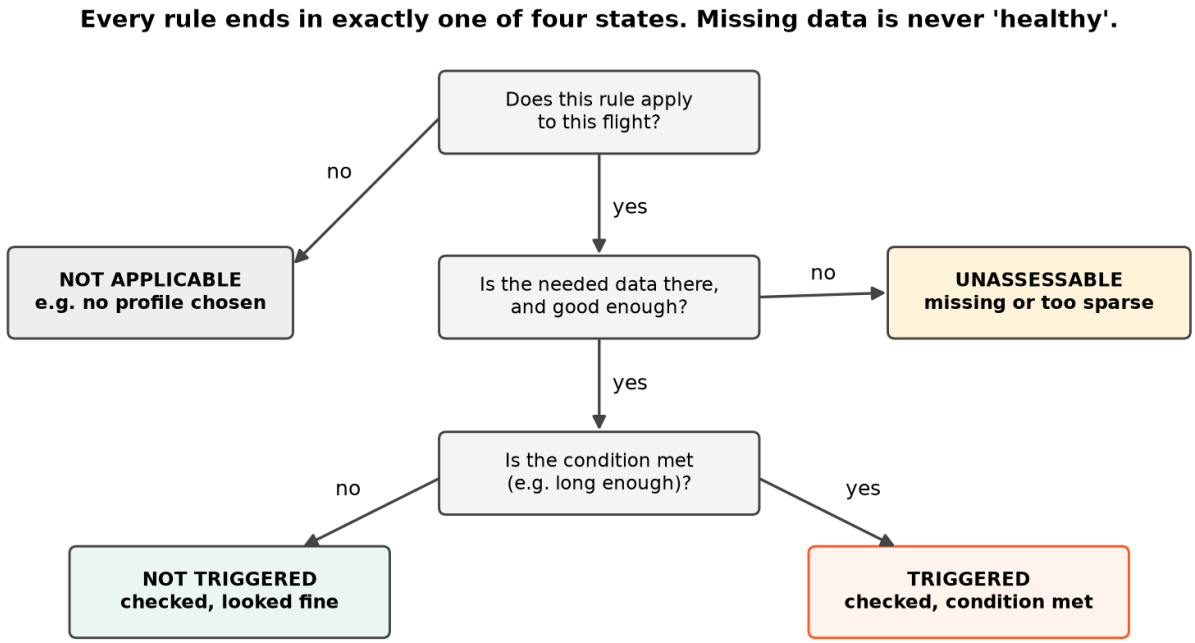


Figure 9.2: Rule states flow

State	Meaning
triggered	The data is there and the condition was met.
not triggered	The data is there and the condition was *not* met.
unassessable	The data needed is missing, or too sparse to decide. **Not the same as healthy.**
not applicable	The rule doesn't apply here, e.g. a profile check when no profile was selected.

Here are real examples from the simulated flights:

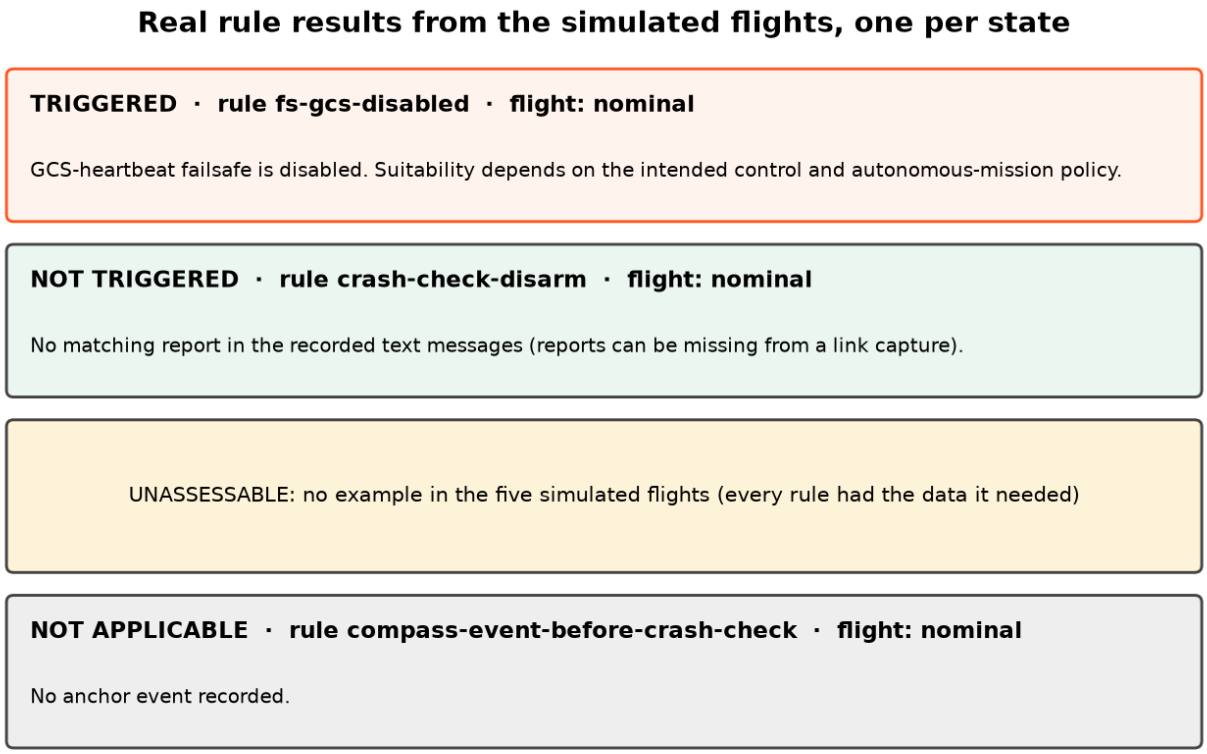


Figure 9.3: Four states examples

Unknown is never healthy

Many tools show a green tick when a check *couldn't run*. AnaTele never does. If a log has no vibration data, the vibration rules say **unassessable**, and any hypothesis that depends on them says so too.

9.4 Counting the five simulated flights

Flight	triggered	not triggered	not applicable
nominal	1	19	2
motor_failure	7	14	1
compass_interference	2	18	2
vibration	5	15	2
radio_failsafe	2	18	2

The one rule that triggers even in the nominal flight is `fs-gcs-disabled`. It's a configuration observation (the ground-station failsafe is turned off), not a fault. That's why rules have **severities** and are kept separate from **hypotheses**.

Chapter 10

Hypotheses

Rules say **what was observed**. Hypotheses ask **what could explain it**. AnaTele ranks the hypotheses only for episodes that end in a critical event (an autopilot crash report). In the five simulated flights that's only the motor-failure flight.

10.1 The four built-in hypotheses

Hypothesis	Supported by (family, weight)	Counted against it
Mechanical / thrust or frame configuration	tracking-loss (control, 1) · motor floor saturation (propulsion, 1) · motor imbalance (propulsion, 0.75) · motor ceiling saturation (propulsion, 0.5)	–
Estimator / magnetic inconsistency	compass event before crash check (estimator, 1) · EKF compass innovation (magnetic, 1) · compasses inconsistent reported (magnetic, 0.25)	compass event flagged as coming after the upset (1)
Vibration-driven estimation degradation	persistent vibration (inertial, 1) · clipping (inertial, 1)	vibration *checked and fine* (0.5) · clipping *checked and fine* (0.5)
Electrical / power degradation	no support rules yet in this pack version	blocked if `power-monitoring-gap` fires

10.2 The score

$$\text{score} = \sum_{\text{families } f} \min\Big(1.5,\ \sum_{\text{triggered rules in } f} w\Big) \ - \sum_{\text{contradictions}} w$$

The **family cap of 1.5** matters. Three rules that all measure motor commands are not three independent pieces of evidence. Capping each family stops one kind of evidence from dominating. Evidence from **different** families (say, *control* and *propulsion*) does add up.

Label	When
strong support	score ≥ 2
some support	score ≥ 1
weak support	score > 0
mixed support	there is both support and contradiction
limited or contradicting evidence	score ≤ 0
unassessable	none of its rules could be checked (or a blocking rule fired)

10.3 Worked example: the motor-failure flight

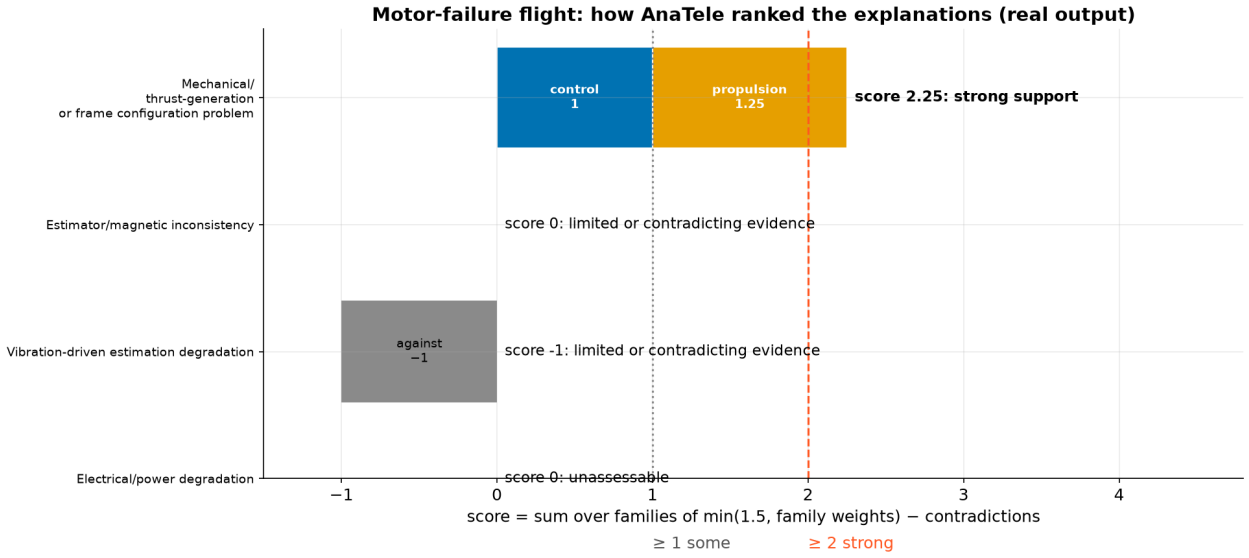


Figure 10.1: Hypothesis scoring

- **Thrust/configuration**: control family = 1 (tracking-loss). Propulsion family = 0.75 + 0.5 = 1.25 (imbalance + ceiling), under the cap. Score = 1 + 1.25 = **2.25**, so **strong support**.
- **Vibration**: nothing triggered, and both vibration rules were *checked and fine*, giving −0.5 − 0.5 = **−1.0**: *limited or contradicting evidence*. Checking and finding nothing counts **against** an explanation.
- **Magnetic**: score **0**. The post-crash "Compasses inconsistent" message doesn't count, because it isn't from the flight's own time window.
- **Power**: **unassessable**. The current pack has no rule that can support it, so it's listed as a reminder of what the log *can't* tell you. Confirming it needs battery voltage/current logs.

Toggle the evidence yourself:

Interactive

This page has an interactive *hypothesis* widget in the web version: `https://avinashistoocool.github.io/AnaTele/learn/09-hypotheses/`

This is a screening order, not a probability

A score of 2.25 doesn't mean "90 % likely". The weights are written-down expert judgements, and the list tells you **what to inspect first**. Each hypothesis also lists the *discriminating evidence* that would confirm or rule it out. For thrust problems that's ESC RPM/current telemetry, a props-off motor test and a physical inspection.

Chapter 11

Missions and geometry

11.1 Where did it go?

An **AUTO mission** is a list of numbered items, usually waypoints (latitude, longitude, altitude). The ground station uploads them, the autopilot flies them in order and reports `MISSION_ITEM_REACHED` as it gets to each one.

AnaTele rebuilds the uploaded plan from the log's own messages. Then, for each waypoint, it checks whether the drone actually got there.

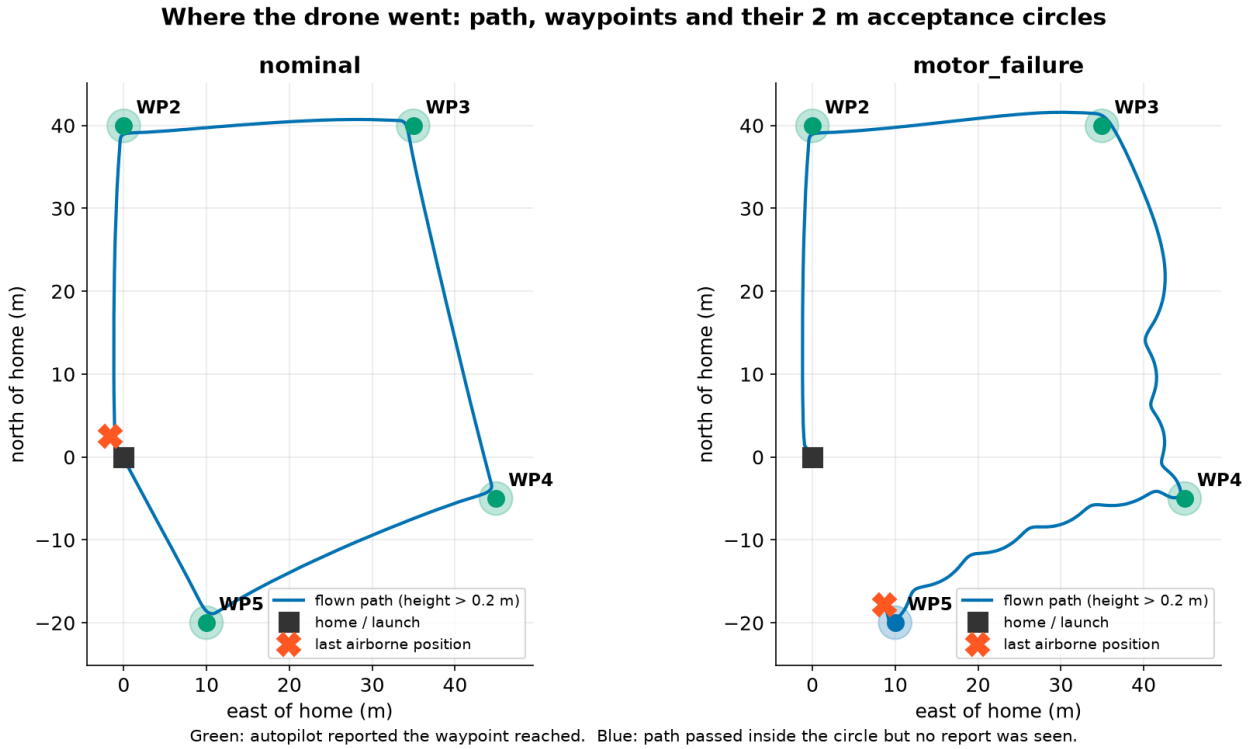


Figure 11.1: Mission map

The simulated mission is a takeoff to 10 m, four waypoints and a return-to-launch. In the nominal flight every waypoint is reported reached. In the motor-failure flight:

Item	Status	Nearest recorded position
WP2–WP4	`reported_complete`	0.97, 0.97, 0.70 m
WP5	`geometry_consistent_with_completion`	0.32 m, but no “reached” report before the crash
RTL	`completion_not_observed`	(not a positional item)

11.2 Distance on a sphere

GPS gives latitude and longitude in degrees, but we need metres. On a sphere of radius R, the shortest distance between two points is along the surface. The **haversine formula** gives it:

$$h = \sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2\frac{\Delta\lambda}{2}, \qquad d = 2R \arcsin\sqrt{h}$$

with φ = latitude, λ = longitude (in radians) and $R = 6\,371\,008.8$ m (the mean Earth radius).

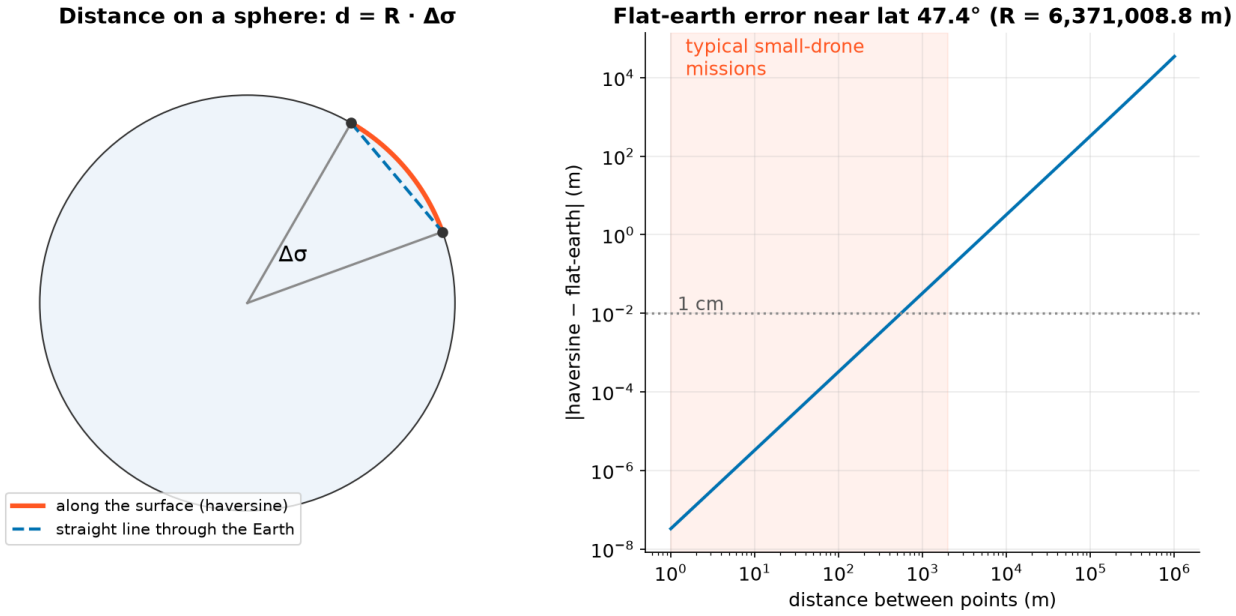


Figure 11.2: Haversine

Over a few kilometres the flat-earth shortcut is accurate to millimetres, far smaller than GPS error. AnaTele uses haversine anyway because it’s correct everywhere, including long survey flights and high latitudes.

Interactive

This page has an interactive *waypoint* widget in the web version: `https://avinashistoocool.github.io/AnaTele/learn/10-missions-and-geometry/`

11.3 When does a waypoint count as reached?

ArduPilot treats a waypoint as reached when the drone is within `WP_RADIUS_M` (2 m in the simulation). GPS has its own uncertainty, so AnaTele allows

$$d_{\text{nearest}} \le \texttt{WP_RADIUS_M} + h_{\text{acc}}$$

where h_{acc} is the GPS horizontal accuracy at that moment. Passing this test is labelled *consistent with completion*, not *completed*: being close isn't proof.

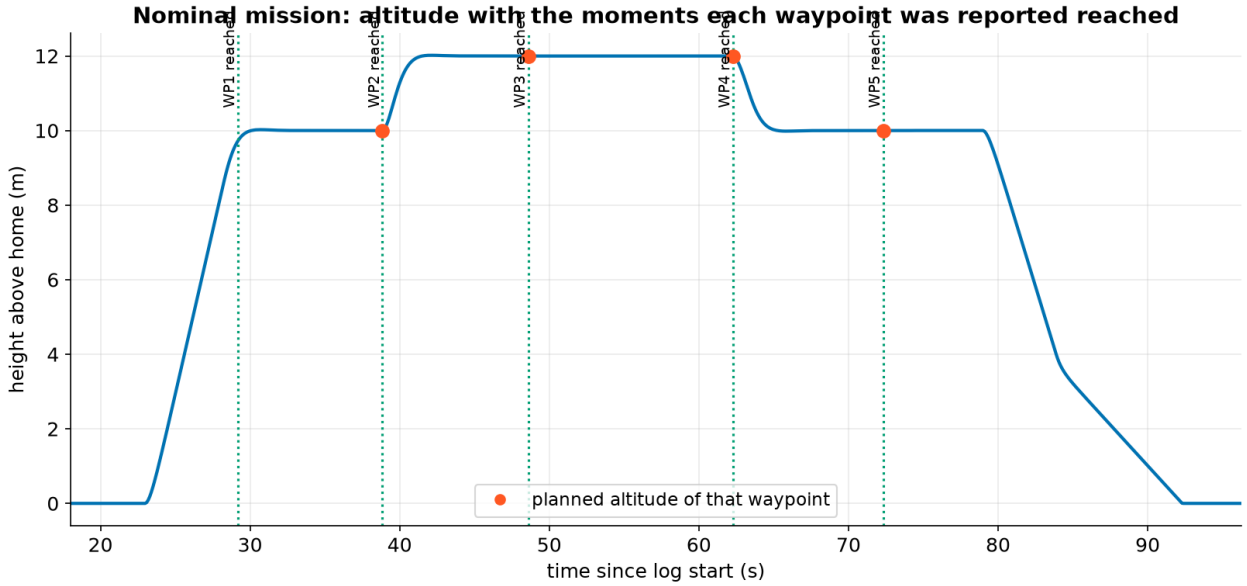


Figure 11.3: Altitude profile

11.4 Drawing the path

The 3D replay in the desktop app draws the flown path with the waypoints. To keep drawing fast it uses at most 6,000 points, keeping every k-th sample with $k = \lceil n/6000 \rceil$:

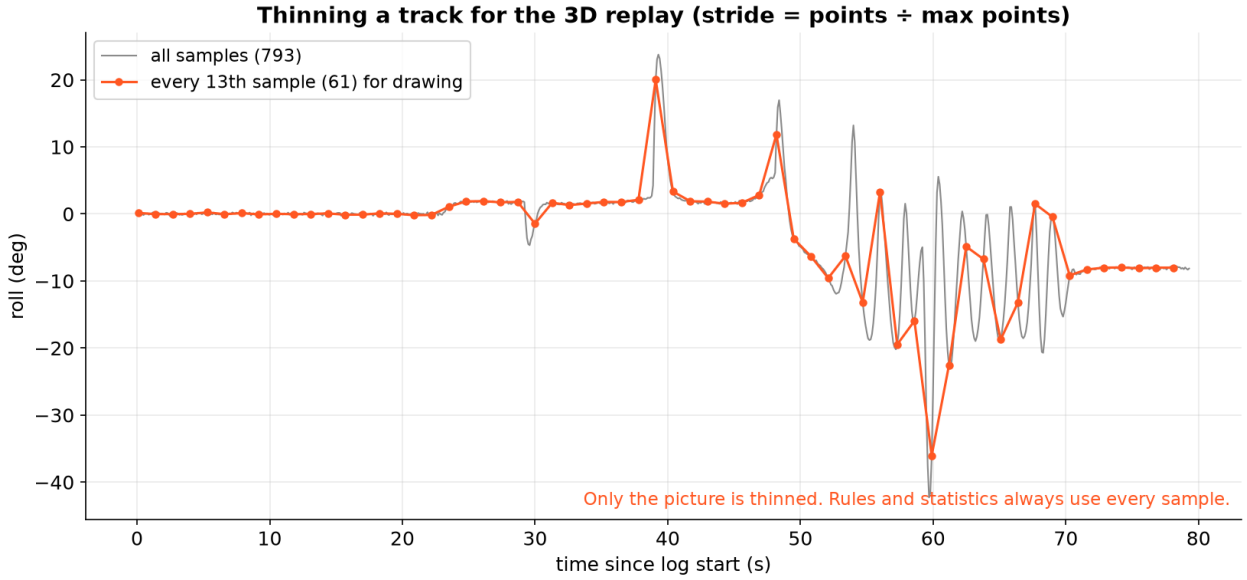


Figure 11.4: Downsampling

Note

Thinning only affects the **picture**. Every rule, statistic and distance above uses every sample.

Chapter 12

The simulator

Every chart in this course came from AnaTele's built-in flight simulator. It exists so that you can **learn and test without crashing a real drone**. Each scenario has a known cause, so you can check that the analysis finds it.

```
anatele simulate --out simulations          # all five scenarios
anatele simulations/sim_motor_failure.tlog  # analyse one
```

12.1 How it works

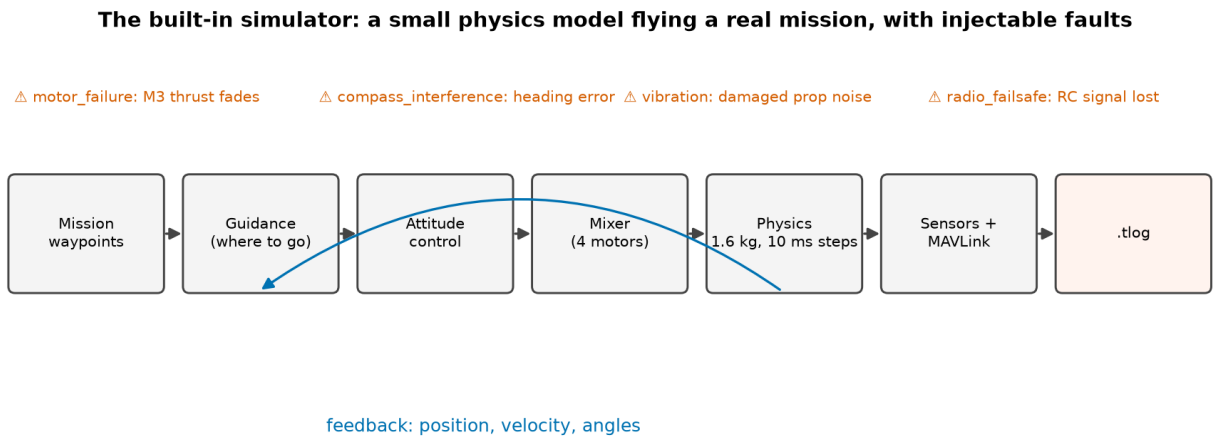


Figure 12.1: Simulator block diagram

Every **10 ms** the simulator:

1. **Guidance** picks a target velocity towards the active waypoint.
2. **Attitude control** turns that into desired roll and pitch (capped at 0.52 rad ≈ 30°), and a yaw target.
3. The **mixer** (chapter 6) turns throttle and torques into four motor commands between 0.15 and 0.95.
4. **Physics** turns commands into thrust and torque. It then integrates Newton's laws for a 1.6 kg quad with 0.18 m arms, drag and a gusting wind.

5. **Sensors and MAVLink**: every 100 ms it writes `ATTITUDE`, `GLOBAL_POSITION_INT`, `SERVO_OUTPUT_RAW` and more (and slower health messages every 500 ms) into a genuine `.tlog`, byte for byte as Mission Planner would.

The thrust per motor is

$$F_i = h_i \cdot F_{\max} \cdot u_i, \qquad F_{\max} = \frac{mg}{4 \cdot 0.38} \approx 10.3 \text{ N},$$

so at the hover output 0.38, four healthy motors exactly hold the weight. h_i is the motor's **health** (1 = perfect).

12.2 The five scenarios

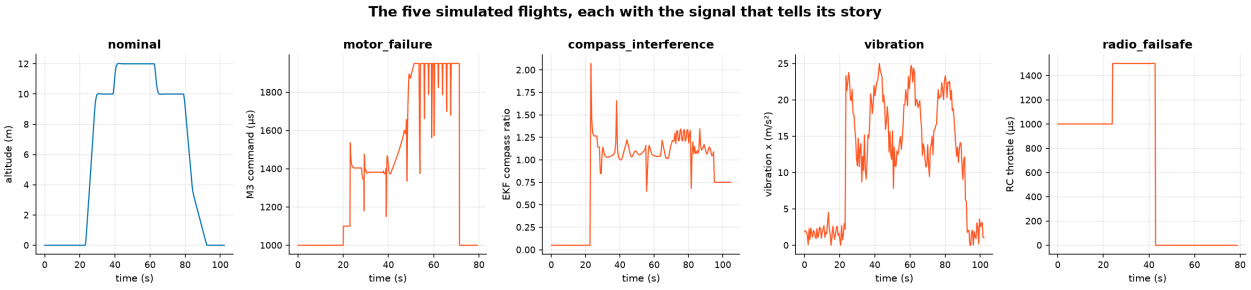


Figure 12.2: Scenarios overview

Scenario	What is injected	What AnaTele finds
`nominal`	nothing	only the configuration note `fs-gcs-disabled`
`motor_failure`	motor 3 health $\max(0.3, 1 - 0.06(t - t_f))$ from 2 s after WP3 becomes active	tracking loss, ceiling saturation, imbalance; *thrust/configuration: strong support*
`compass_interference`	heading error that grows with motor current	EKF compass innovation above 1 for 24.5 s
`vibration`	random torque noise (a damaged prop)	persistent vibration, 283 clips
`radio_failsafe`	RC signal lost mid-mission	pre-arm *radio failsafe* reports

Make your own

`Simulator(scenario, seed, mission=[...], wind=(n, e))` in `telemetry_platform/simulator.py` takes your own mission and wind. Different seeds give different noise but the same physics.

Careful

The simulator is a teaching and testing model, not a certified flight dynamics model. Real crashes are messier. Use it to understand the method, not to calibrate thresholds for a real aircraft.

Chapter 13

Sharing with .ant files

An investigation is often a team effort. The pilot, the builder and a reviewer may all need to see the same thing. Instead of sending a raw log and hoping everyone uses the same version of every tool, AnaTele saves the **whole analysis** as one file: `.ant`.

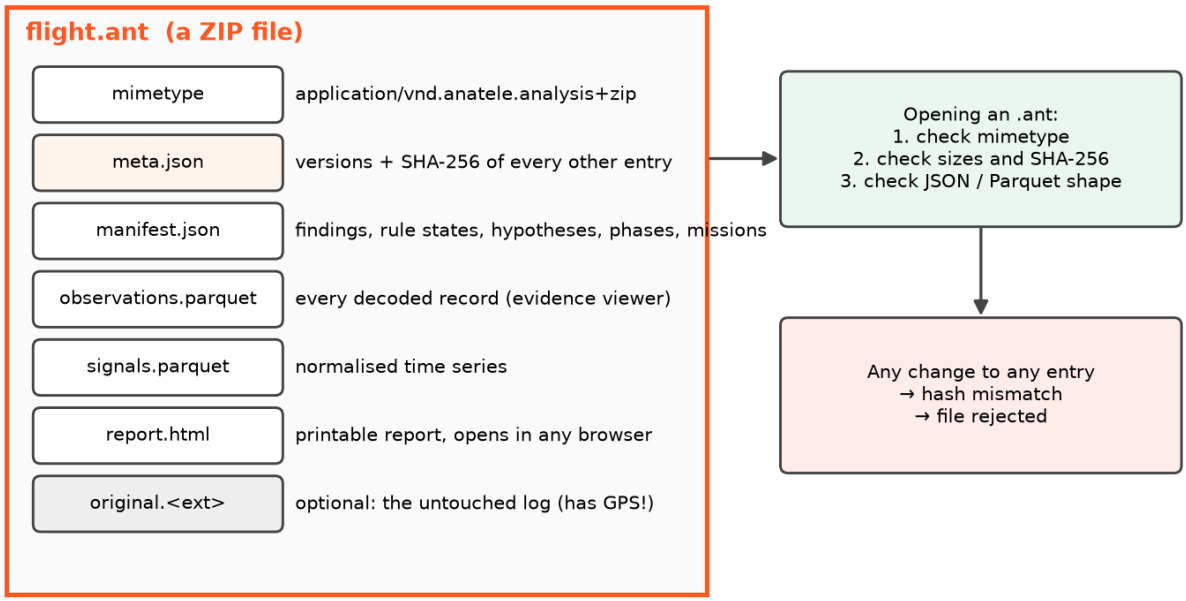


Figure 13.1: .ant file

13.1 What is inside

An `.ant` is an ordinary ZIP file. You can open it with any unzip tool, but AnaTele checks it strictly:

1. `mimetype` must be exactly `application/vnd.anatele.analysis+zip`.
2. `meta.json` lists the **SHA-256** hash of every other entry. A SHA-256 hash is a 64-character fingerprint: change one byte of a file and the fingerprint changes completely.
3. Each entry is size-limited, hashed and compared. The JSON and Parquet structure is validated **before** anything is shown.

Any mismatch and the file is rejected. A modified analysis can't pass as the original.

13.2 Saving and opening

Desktop **File** → **Save Analysis As (.ant)** (Ctrl+S). Open with **File** → **Open** or drag and drop.

Command line

```
anatele export flight.tlog -o flight.ant            # analysis only
anatele export flight.tlog -o flight.ant --include-original
anatele analyse flight.ant                           # print the summary
```

Web / API `GET /api/logs/{id}/export.ant` to download, `POST /api/analyses` to import.

Privacy

`-include-original` embeds the untouched log, and that contains **GPS positions** (where you fly, often where you live). Leave it out unless the receiver needs to re-run the analysis or inspect raw bytes.

13.3 The report inside

`report.html` is a self-contained, printable report: findings, hypotheses, rule states, phases and the mission table. It opens in any browser, so someone without AnaTele can still read the result.

You've finished the course. Next: open a real log in the desktop app, use the Usage guide for the controls and the Diagnostics reference for every rule.

Appendix A

Glossary

Term	Meaning
ArduPilot	Open-source autopilot firmware for drones, planes, rovers and boats.
Armed	Motors are allowed to spin. An *episode* is one armed period.
Attitude	Orientation of the drone: roll, pitch and yaw.
Baseline altitude	Median altitude over the first 2 s after arming, treated as "ground".
Ceiling / floor	The highest / lowest motor command allowed (`MOT_SPIN_MAX` / `MOT_SPIN_MIN`).
Clipping	An accelerometer reading hit the sensor's maximum, so the true value is unknown.
DataFlash log	On-board log (`.bin`/`.log`) written to the flight controller's SD card.
EKF	Extended Kalman Filter, ArduPilot's sensor-fusion estimator.
Epoch	A stretch of a log in which the clock only moves forward (split at reboots).
Event guard	The pre-event window ends 3 s before an autopilot crash report.
Family (evidence)	A group of related rules; its contribution to a hypothesis is capped at 1.5.
Haversine	Formula for distance between two latitude/longitude points on a sphere.
Hypothesis	A possible explanation, scored by evidence for and against. Not a probability.
Innovation / test ratio	How far a sensor reading is from the EKF's prediction, scaled so 1.0 is the limit.
MAVLink	The message protocol between drones and ground stations.
Mixer	Turns throttle/roll/pitch/yaw demands into individual motor commands.
Persistence	A condition must hold continuously for a minimum time to count.
Pre-event window	From takeoff to just before the upset or crash report; where rules look for causes.
PWM	Pulse width (µs) sent to a motor controller; here 1000 = off, 2000 = full.
Rule pack	The versioned YAML file of all rules and hypotheses.
Rule state	triggered / not triggered / unassessable / not applicable.
Saturation	A motor stuck at its ceiling or floor, so the controller has no headroom left.
Source	One MAVLink sender (system id : component id), e.g. `1:1`.
STATUSTEXT	Human-readable text messages from the autopilot (e.g. "Crash: Disarming").
Tilt error	Angle between desired and actual thrust direction.
.tlog	Telemetry log saved by the ground station from the radio link.
.ant	AnaTele's shareable, hash-verified analysis file.
Unassessable	The data needed to check something is missing or too sparse. Never "healthy".
WP_RADIUS_M	How close the drone must get for a waypoint to count as reached.